

```
/*
=====
=====
    ranking.js – Chart Ranking Algorithm
    -----
    -----
    Calculates a transparent, data-driven
    ranking score for each song based on:

    1. Views (40% weight) – most popular
    metric
    2. Likes (30% weight) – quality signal,
    harder to fake than views
    3. Comments (20% weight) – audience
    engagement & discussion
    4. Upload Recency (10% weight) – boost
    newer songs, avoid stale charts

    Each component is normalized to 0-100,
    then weighted and summed.
    Songs are ranked by total score, ties
    broken by views descending.

    This mimics Billboard's approach: hits
    need BOTH reach (views) AND
    engagement (likes + comments), with a
    small freshness boost.

=====
===== */
```

```

window.HVC = window.HVC || {};

/**
 * Ranking & Filtering Service
 */
window.HVC.Ranking = (() => {

    /**
     * Normalize a value to 0-100 using min-
     max normalization.
     * If all values are 0, returns 0 (avoid
     NaN).
     */
    function normalize(value, min, max) {
        if (max === min || max === 0) return 0;
        return Math.max(0, Math.min(100,
            ((value - min) / (max - min)) * 100));
    }

    /**
     * Calculate days since upload
     */
    function daysSinceUpload(uploadedAt) {
        const now = new Date();
        const diff = now.getTime() -
            uploadedAt.getTime();
        return diff / (1000 * 60 * 60 * 24);
    }

    /**
     * Calculate recency score: newer songs
     score higher, but old hits don't drop to 0.
     * Formula: if < 7 days old → 100

```

```

*           if < 30 days old → 80-100
*           if < 90 days old → 50-80
*           if >= 90 days old → 0-50
* This means brand-new songs get a
boost, but songs from last quarter stay
relevant.
*/
function recencyScore(uploadedAt) {
  const days =
daysSinceUpload(uploadedAt);

  if (days <= 7) return 100;
  if (days <= 30) return 100 - ((days -
7) / 23) * 20; // 80-100
  if (days <= 90) return 80 - ((days -
30) / 60) * 30; // 50-80
  return Math.max(0, 50 - ((days - 90) /
100) * 50); // 0-50, asymptotic
}

/**
* Calculate engagement score: ratio of
(likes + comments*0.5) to views.
* Accounts for the fact that comments
are rarer but valuable.
* Returns 0-100.
*/
function engagementScore(views, likes,
comments) {
  if (views === 0) return 0;

  // Weighted engagement metric: likes
count more than comments
  const engagement = (likes * 0.7 +

```

```

comments * 0.3) / views;

    // Normalize to 0-100 using expected
    engagement benchmarks
    // Good engagement is 2-5%, excellent
    is 5%+
    return Math.min(100, engagement *
2000); // 0.5% → 10, 5% → 100
    }

/**
 * Main ranking score calculation
 * Weights:
 *   Views: 40%
 *   Likes: 30%
 *   Comments: 20%
 *   Recency: 10%
 */
function calculateScore(song, allSongs =
[]) {
    // Find min/max for normalization
    (across all songs in the set)
    let maxViews = 1, maxLikes = 1,
maxComments = 1;

    if (allSongs.length > 0) {
        maxViews = Math.max(...allSongs.map(s
=> s.views)) || 1;
        maxLikes = Math.max(...allSongs.map(s
=> s.likes)) || 1;
        maxComments =
Math.max(...allSongs.map(s => s.comments))
|| 1;
    }
}

```

```
        // Component scores (0-100 each)
        const viewsScore =
normalize(song.views, 0, maxViews);
        const likesScore =
normalize(song.likes, 0, maxLikes);
        const commentsScore =
normalize(song.comments, 0, maxComments);
        const recency =
recencyScore(song.uploadedAt);

        // Weighted total (40 + 30 + 20 + 10 =
100)
        const totalScore = (
            viewsScore * 0.40 +
            likesScore * 0.30 +
            commentsScore * 0.20 +
            recency * 0.10
        );

        return {
            total: totalScore,
            components: {
                views: viewsScore,
                likes: likesScore,
                comments: commentsScore,
                recency: recency,
                engagement:
engagementScore(song.views, song.likes,
song.comments)
            }
        };
    }
}
```

```

/**
 * Score and sort songs.
 * Songs with a manually-set rank (from
the admin "Rank" field) are pinned
 * to that position, in order. Every
other song fills in around them,
 * sorted by computed score. Final
display rank (1..n) is reassigned
 * after combining both groups so
numbering has no gaps.
 */
function rankSongs(songs) {
  // Calculate score for each song
  const scored = songs.map(song => ({
    ...song,
    score: calculateScore(song, songs)
  }));

  const manuallyRanked = scored
    .filter(s => s.manualRank !== null &&
s.manualRank !== undefined)
    .sort((a, b) => a.manualRank -
b.manualRank);

  const autoRanked = scored
    .filter(s => s.manualRank === null ||
s.manualRank === undefined)
    .sort((a, b) => {
      if (b.score.total !==
a.score.total) {
        return b.score.total -
a.score.total;
      }
      return b.views - a.views;
    });

```

```

    });

    const combined = [...manuallyRanked,
...autoRanked];

    // Add final display rank
    return combined.map((song, index) => ({
        ...song,
        rank: index + 1
    }));
}

/**
 * Filter songs by date range
 */
function filterByDateRange(songs, range)
{
    const now = new Date();
    let startDate;

    switch (range) {
        case 'today':
            startDate = new Date(now);
            startDate.setHours(0, 0, 0, 0);
            break;
        case 'week':
            startDate = new Date(now);
            startDate.setDate(now.getDate() -
7);
            break;
        case 'month':
            startDate = new Date(now);
            startDate.setMonth(now.getMonth() -
1);

```

```

        break;
    case 'year':
        startDate = new Date(now);

    startDate.setFullYear(now.getFullYear() -
1);

        break;
    default:
        return songs; // No filter
    }

    return songs.filter(song => new
Date(song.uploadedAt) >= startDate);
}

/**
 * Get "new release" songs (uploaded
within the window)
 */
function getNewReleases(songs, windowDays
= 21) {
    const cutoff = new Date();
    cutoff.setDate(cutoff.getDate() -
windowDays);
    return songs.filter(s => s.uploadedAt
>= cutoff);
}

/**
 * Get "fastest rising" songs (strong
engagement, uploaded recently)
 */
function getFastestRising(songs,
windowDays = 45) {

```



```

    const cutoff = new Date();
    cutoff.setDate(cutoff.getDate() -
windowDays);

    return songs
      .filter(s => s.uploadedAt >= cutoff)
      .sort((a, b) => {
        // Sort by engagement (likes +
comments) relative to views
        const engageA = (a.likes +
a.comments * 0.5) / (a.views || 1);
        const engageB = (b.likes +
b.comments * 0.5) / (b.views || 1);
        return engageB - engageA;
      });
  }

  /**
   * Get most viewed songs (all-time)
   */
  function getMostViewed(songs, limit =
100) {
    return [...songs].sort((a, b) =>
b.views - a.views).slice(0, limit);
  }

  /**
   * Search songs by title, artist, or
label
   */
  function search(songs, query) {
    if (!query || query.trim() === "")
return songs;

```

```

        const q = query.toLowerCase().trim();
        return songs.filter(song => {
            const titleMatch =
song.title.toLowerCase().includes(q);
            const artistMatch =
song.channelName.toLowerCase().includes(q);
            const labelMatch = (song.label ||
"").toLowerCase().includes(q);
            return titleMatch || artistMatch ||
labelMatch;
        });
    }

    /**
     * Get top artists by number of songs in
top N
     */
    function getTopArtists(rankedSongs, topN
= 100) {
        const artistMap = new Map();

        rankedSongs.slice(0, topN).forEach(song
=> {
            if (!artistMap.has(song.channelName))
{
                artistMap.set(song.channelName, {
                    name: song.channelName,
                    songs: [],
                    totalViews: 0
                });
            }
            const artist =
artistMap.get(song.channelName);
            artist.songs.push(song);

```

```

        artist.totalViews += song.views;
    });

    // Convert to array and sort by song
    count, then by total views
    const artists =
    Array.from(artistMap.values());
    artists.sort((a, b) => {
        if (b.songs.length !==
    a.songs.length) {
        return b.songs.length -
    a.songs.length;
        }
        return b.totalViews - a.totalViews;
    });

    return artists;
}

/**
 * Get recently added songs (by upload
    date, descending)
 */
function getRecentlyAdded(songs, limit =
    50) {
    return [...songs]
        .sort((a, b) => b.uploadedAt -
    a.uploadedAt)
        .slice(0, limit);
}

/**
 * Format large numbers for display
    (e.g., 1500000 → "1.5M")

```

```

*/
function formatNumber(num) {
  if (num >= 1_000_000) {
    return (num /
1_000_000).toFixed(1).replace(/\.0+$/, "")
+ "M";
  }
  if (num >= 1_000) {
    return (num /
1_000).toFixed(1).replace(/\.0+$/, "") +
"K";
  }
  return num.toString();
}

/**
 * Format date for display (e.g., "Jan
15, 2024")
 */
function formatDate(date) {
  return new
Date(date).toLocaleDateString("en-US", {
    year: "numeric",
    month: "short",
    day: "numeric"
  });
}

/**
 * Get relative time (e.g., "3 days ago",
"2 weeks ago")
 */
function getRelativeTime(date) {
  const now = new Date();

```

```

    const seconds = Math.floor((now - date)
/ 1000);
    let interval = Math.floor(seconds /
31536000);

    if (interval >= 1) return interval ===
1 ? "1 year ago" : `${interval} years ago`;
    interval = Math.floor(seconds /
2592000);
    if (interval >= 1) return interval ===
1 ? "1 month ago" : `${interval} months
ago`;
    interval = Math.floor(seconds / 86400);
    if (interval >= 1) return interval ===
1 ? "1 day ago" : `${interval} days ago`;
    interval = Math.floor(seconds / 3600);
    if (interval >= 1) return interval ===
1 ? "1 hour ago" : `${interval} hours ago`;
    interval = Math.floor(seconds / 60);
    if (interval >= 1) return interval ===
1 ? "1 minute ago" : `${interval} minutes
ago`;
    return "just now";
}

// Public API
return {
  calculateScore,
  rankSongs,
  filterByDateRange,
  getNewReleases,
  getFastestRising,
  getMostViewed,
  search,

```

```
    getTopArtists,  
    getRecentlyAdded,  
    formatNumber,  
    formatDate,  
    getRelativeTime,  
    daysSinceUpload  
  };  
})();
```